



Module 2: Callbacks and the Event Loop



Module Objective

By the end of this module, students should be able to:

- Understand what **Callbacks** are and how they work.
- Describe how the **Event Loop** manages concurrency in JavaScript.
- Differentiate between **Synchronous** and **Asynchronous** code.
- Write simple programs using callbacks and observe how the event loop executes them.



Keywords and Definitions

Keyword	Definition
Callback Function	A function passed as an argument to another function, to be executed later when a task completes.
Asynchronous	Code that doesn't block the execution of other tasks while waiting for a result.
Synchronous	Code that executes line-by-line, waiting for one task to finish before moving to the next.
Event Loop	A mechanism that continuously checks if the call stack is empty, then pushes queued tasks (like callbacks) for execution.
Call Stack	The structure that keeps track of function calls — last function in is executed first (LIFO).
Callback Queue (Task Queue)	A waiting area for asynchronous tasks (e.g., <code>setTimeout</code> , <code>fetch</code>) until the call stack is free.
Web API	Browser-provided features (like <code>setTimeout</code> , <code>fetch</code> , DOM events) that handle asynchronous tasks outside the main thread.



1. Understanding Callbacks



Explanation:

A **callback** is a function that is executed **after another function finishes**.

This is how JavaScript handles tasks that take time — such as waiting for user input, API calls, or timers — **without blocking** the main thread.



Example 1: Simple Callback

```
function greet(name, callback) {  
  console.log("Hello, " + name);  
  callback();  
}
```

```
function sayGoodbye() {  
  console.log("Goodbye!");  
}
```

```
greet("Joel", sayGoodbye);
```



Explanation:

- The function `greet()` takes a name and another function as an argument.
- After greeting, it calls the `callback()` function.



Output:

```
Hello, Joel  
Goodbye!
```



Example 2: Asynchronous Callback

```
console.log("Start");
```

```
setTimeout(() => {  
  console.log("This runs later!");  
}, 2000);
```

```
console.log("End");
```

Explanation:

- `setTimeout()` is handled by the **Web API**, not the main thread.
- After 2 seconds, the callback is pushed to the **Callback Queue**, and the **Event Loop** checks when to execute it.

Output:

Start

End

This runs later!

This shows **asynchronous** behavior — JavaScript doesn't wait for `setTimeout` to finish before moving to the next line.

2. Understanding the Event Loop

Explanation:

The **Event Loop** is what makes JavaScript *appear* concurrent.
It constantly checks:

1. Is the **call stack** empty?
2. Are there **callbacks** waiting in the queue?
3. If yes, push the next task onto the stack and execute it.

Example 3: Event Loop Visualization

```
console.log("Task 1");
```

```
setTimeout(() => {  
  console.log("Task 2 (Async)");  
}, 0);
```

```
console.log("Task 3");
```

Output:

Task 1

Task 3

Task 2 (Async)

Explanation:

- Even though the timeout is 0ms, it still goes to the **Callback Queue**.
- The **Event Loop** waits until the stack is empty before executing it.



3. Hands-On Practical



Practical 1: Custom Callback

Objective: Write a program that simulates loading data using a callback.

```
function loadData(callback) {  
  console.log("Fetching data...");  
  setTimeout(() => {  
    console.log("Data fetched successfully!");  
    callback();  
  }, 2000);  
}
```

```
function showData() {  
  console.log("Displaying data on screen...");  
}
```

```
loadData(showData);
```

Expected Output:

Fetching data...

Data fetched successfully!

Displaying data on screen...

Explanation:

`loadData()` simulates an async operation, and `showData()` runs only after it finishes — demonstrating callback-based concurrency.

Practical 2: Understanding Callback Queue

```
console.log("A");

setTimeout(() => {
  console.log("B (from Callback Queue)");
}, 0);

Promise.resolve().then(() => {
  console.log("C (Microtask Queue)");
});

console.log("D");
```

✅ Expected Output:

```
A
D
C (Microtask Queue)
B (from Callback Queue)
```

💡 Explanation:

- The **microtask queue** (Promise) runs before the **callback queue** (setTimeout).
- This introduces the concept of **task priority**.

4. Classwork / Exercises

Exercise 1:

Write a program that:

1. Logs "Start"
2. Uses `setTimeout()` to log "Loading..." after 2 seconds
3. Logs "Process Complete!" right after the timeout callback finishes.

 *Hint:* Use callbacks to control the flow.



Exercise 2:

Explain, in your own words:

1. What happens when a function with `setTimeout()` is called?
2. How does the Event Loop know when to execute the callback?
3. Why doesn't `setTimeout()` block the rest of the code?



5. Summary

Concept	Meaning
Callback	Function executed after another finishes.
Asynchronous Code	Non-blocking code that continues running while waiting.
Event Loop	Mechanism that manages the execution of async tasks.
Call Stack	Keeps track of what's currently running.
Callback Queue	Stores async callbacks waiting to be executed.
Web API	Handles tasks like timers and HTTP requests outside the JS thread.

🎲 Module 2 Recap

In this module, we learned:

- The concept of **Callbacks** and how they control async flow.
- How the **Event Loop** works behind the scenes.
- The difference between **Synchronous** and **Asynchronous** behavior.
- How **Web APIs** and **Queues** enable concurrency in JavaScript.